

Error-detecting and error-correcting codes

Section 5.4 of *Numbers, Groups and Codes*

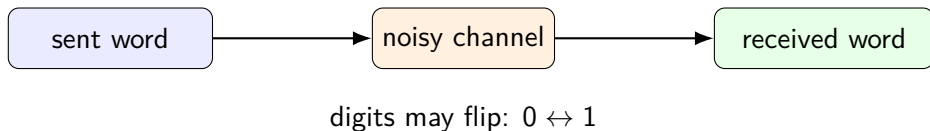
Applied Algebra
Instructor: Tuan Tran

Motivation and binary codes

Add redundancy so that errors become visible

The problem: messages can change in transit

Messages sent over a channel may be distorted.



Examples: electrical noise, corrupted storage, and long-distance transmission.

Goal: detect errors, and when possible correct them without retransmission.

What these codes are not

There are two different coding goals.

Cryptographic codes

- keep information secret;
- decoding should be hard without a key;
- Chapter 1: public key codes.

Error-correcting codes

- keep information accurate;
- decoding should be systematic;
- Section 5.4: add check digits.

Here the issue is reliability, not secrecy.

Binary alphabet and binary words

From now on we work with binary information.

Let

$$\mathbb{B} = \mathbb{Z}_2 = \{0, 1\}$$

with addition and multiplication modulo 2.

A **word of length** n is a string of n binary digits.

Examples of words in $\mathbb{B}^4$:

0001, 1110, 0000.

We regard $\mathbb{B}^n$ as an Abelian group under componentwise addition.

Addition in $\mathbb{B}^n$

In $\mathbb{B}$, we have

$$1 + 1 = 0.$$

So every word is its own inverse:

$$w + w = 0.$$

Example:

$$010101 + 101000 = 111101.$$

A 1 in $v + w$ marks a position where v and w differ.

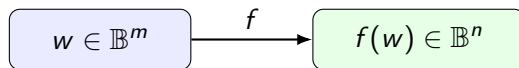
Coding functions

Suppose original messages are words of length m .

A **coding function** is an injective function

$$f : \mathbb{B}^m \longrightarrow \mathbb{B}^n, \quad n > m.$$

Instead of sending w , we send $f(w)$.



The words in the image of f are called **codewords**.

Why f should be injective

If two different messages have the same codeword, they cannot be distinguished after transmission.

$$\begin{aligned}w_1 &\neq w_2, \\ f(w_1) &= f(w_2)\end{aligned}$$

would mean the receiver cannot know whether w_1 or w_2 was sent.

So for a useful code:

$$f(w_1) = f(w_2) \quad \Rightarrow \quad w_1 = w_2.$$

Extra digits are allowed; loss of information is not.

Warm-up: ISBN check digit

The ISBN example uses a check digit modulo 11.

Given nine digits $a_1 a_2 \cdots a_9$, form

$$N = 10a_1 + 9a_2 + 8a_3 + \cdots + 2a_9.$$

Reduce N modulo 11 and choose a final check digit so that a valid ISBN satisfies a fixed congruence.

If one digit changes by k in position i , then N changes by

$$k(11 - i).$$

Since 11 is prime and $1 \leq 11 - i \leq 10$, this change is not divisible by 11.

Therefore a single transcription error is detected.

Example 1: parity check code

Define

$$f : \mathbb{B}^m \longrightarrow \mathbb{B}^{m+1}, \quad f(w) = wx,$$

where x is chosen so that $f(w)$ has an even number of 1s.

For $m = 3$:

w	000	001	010	011	100	101	110	111
$f(w)$	0000	0011	0101	0110	1001	1010	1100	1111

A single digit flip changes parity, so the received word is not a codeword.

What parity detects

The parity-check code detects:

1, 3, 5, ... errors.

It does **not** detect:

2, 4, 6, ... errors.

Example:

0000 $\longrightarrow$ 1100

contains two flips, but both words have even parity.

Parity detects a problem but does not locate it.

Example 2: repeat three times

Define

$$f : \mathbb{B}^m \longrightarrow \mathbb{B}^{3m}, \quad f(w) = www.$$

For example,

$$w = 101111$$

becomes

$$f(w) = 101111 \ 101111 \ 101111.$$

This code can detect one or two errors.

It can also correct one error by majority vote among the three blocks.

Majority vote correction

Suppose a word is sent as

$www.$

Break a received word into three blocks:

$abc.$

If at most one error has occurred, then two blocks agree with the original w .

Received blocks	a	b	c
No error	w	w	w
One error	w'	w	w
One error	w	w'	w
One error	w	w	w'

The repeated value is the most likely original word.

The trade-off

More check digits usually mean better reliability, but more transmission cost.

Code	Length sent	Detects	Corrects
Parity check	$m + 1$	odd errors	none
Triple repetition	$3m$	1 or 2 errors	1 error

Later examples will do better than simple repetition.

Good codes spread codewords far apart while using few extra digits.

Weight

The **weight** of a binary word is the number of 1s in it:

$$\text{wt}(w) = \text{number of 1s in } w.$$

Examples:

$$\text{wt}(001101) = 3, \quad \text{wt}(000) = 0, \quad \text{wt}(111) = 3.$$

Weight measures how far a word is from the zero word:

$$\text{wt}(w) = d(w, 0).$$

Distance between words

For $v, w \in \mathbb{B}^n$, define the distance by

$$d(v, w) = \text{wt}(v - w).$$

But in $\mathbb{B}^n$ we have $v - w = v + w$, so

$$d(v, w) = \text{wt}(v + w).$$

This is exactly the number of positions where v and w differ.

Examples:

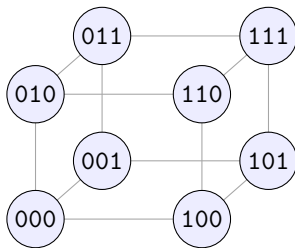
$$d(010101, 101000) = \text{wt}(111101) = 5,$$

$$d(1111, 0110) = \text{wt}(1001) = 2.$$

Hamming graph viewpoint

Words in $\mathbb{B}^n$ can be drawn as vertices of an n -dimensional cube.

Two vertices are joined by an edge if they differ in exactly one coordinate.



Good codewords should be well separated in this graph.

Minimum distance of a code

Let $\mathcal{C}$ be the set of codewords.

The **minimum distance** is

$$d_{\min} = \min\{d(u, v) : u, v \in \mathcal{C}, u \neq v\}.$$

Why this matters:

- one error moves a word distance 1;
- two errors move it distance at most 2;
- in general, k errors move it distance at most k .

Error control is geometry in the Hamming graph.

Theorem 5.4.1: error detection

Theorem. A code detects all errors of size at most k if and only if

$$d_{\min} \geq k + 1.$$

Reason. If a codeword is changed in at most k positions, it lands within distance k of the original codeword.

If all other codewords are at distance at least $k + 1$, the received word cannot be another codeword.

So an error is visible exactly because the received word lies outside $\mathcal{C}$.

Theorem 5.4.2: error correction

Theorem. A code corrects all errors of size at most k if and only if

$$d_{\min} \geq 2k + 1.$$

Uniqueness idea. If r is within distance k of two codewords c and c' , then

$$d(c, c') \leq d(c, r) + d(r, c') \leq 2k.$$

So if $d_{\min} \geq 2k + 1$, this cannot happen.

Correction means nearest-codeword decoding is unambiguous.

Detection and correction at a glance

If the minimum distance is $d_{\min}$, then the code detects up to

$$d_{\min} - 1$$

errors, and corrects up to

$$\left\lfloor \frac{d_{\min} - 1}{2} \right\rfloor$$

errors.

$d_{\min}$	detects up to	corrects up to
2	1	0
3	2	1
4	3	1
5	4	2

Quick checks:

- ① Compute $d(01011, 11001)$.
- ② A code has $d_{\min} = 4$. How many errors does it detect? How many does it correct?
- ③ Why can parity check detect a single error but not correct it?
- ④ In the triple repetition code, why can two errors lead to a wrong correction?